

Batch – T5**Assignment No. – 8****Title – Dynamic Programming Algorithms****Student Name – Sharaneshwar Bharat Punjal****Student PRN – 23520011**

1) You are given an array containing n integers. Your task is to determine the longest increasing subsequence in the array, i.e., the longest subsequence where every element is larger than the previous one.

A subsequence is a sequence that can be derived from the array by deleting some elements without changing the order of the remaining elements.

Input:

The first line contains an integer n : the size of the array.

After this there are n integers $x_1, x_2, \dots, x_n$: the contents of the array.

Output:

Print the length of the longest increasing subsequence.

Constraints:

$1 \leq n \leq 2 * 10^5$

$1 \leq x_i \leq 10^9$

Example

Input:

8

7 3 5 3 6 2 9 8

Output:

4

Brute Force Approach:

1. Generate all possible subsequences of the given array.
2. For each subsequence, check if it is strictly increasing.
3. Keep track of the length of the longest increasing subsequence found.
4. Return the length of the longest subsequence.

Time Complexity: $O(2^n)$, where n is the number of elements in the array, due to generating all possible subsequences.

Dynamic Programming (DP) Approach (Memoization):

1. Create a recursive function that takes the current index and the previous index as parameters.
2. For each index, decide to either include or exclude the current element in the subsequence.
3. Use memoization to store results for already computed states to avoid redundant calculations.
4. Return the length of the longest increasing subsequence found.

Time Complexity: $O(n^2)$, where n is the number of elements in the array.

Dynamic Programming (DP) Approach (Tabulation):

1. Create a DP array where $dp[i]$ represents the length of the longest increasing subsequence ending at index i .

2. Initialize all elements of the DP array to 1, as the minimum length of the LIS ending at each element is 1 (the element itself).
 3. Iterate through the array, and for each element, check all previous elements to see if they can extend the LIS.
 4. Update the DP array accordingly.
 5. Return the maximum value in the DP array.
- Time Complexity: $O(n^2)$, where n is the number of elements in the array.

Binary Search Approach:

1. Create a DP array to store the smallest tail for increasing subsequences of different lengths.
2. Iterate through the input array. For each element, use binary search to find its position in the DP array.
3. If the element is greater than the largest element in the DP array, append it. Otherwise, replace the element at the found position.
4. The length of the DP array will give the length of the longest increasing subsequence.

Time Complexity: $O(n \log n)$, where n is the number of elements in the array, due to the binary search operation.

Code:

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Scanner;

public class Q1 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }

        List<Integer> lis = findLIS(arr, n);
        for (int num : lis) {
            System.out.print(num + " ");
        }
        System.out.println();
        scanner.close();
    }

    private static List<Integer> findLIS(int[] arr, int n) {
        int[] dp = new int[n];
        List<Integer> lis = new ArrayList<>();
        int len = 0;

        for (int x : arr) {
```

```
        int i = Arrays.binarySearch(dp, 0, len, x);
        if (i < 0) i = -(i + 1);
        dp[i] = x;
        if (i == len) len++;
        if (i < lis.size()) {
            lis.set(i, x);
        } else {
            lis.add(x);
        }
    }
    return lis;
}
```

Test Case 1:

12

3 10 2 1 20 4 6 5 7 8 9 30

Output:

1 4 5 7 8 9 30

Test Case 2:

8

7 3 5 3 6 2 9 8

Output:

2 5 6 8

Test Case 3:

20

10 9 2 5 3 7 10 1 18 21 22 23 24 25 26 27 28 29 30 31 32

Output:

2 3 7 18 21 22 23 24 25 26 27 28 29 30 31 32

Test Case 4:

10

3 2 6 5 0 3 7 8 1 4

Output:

0 1 4 8

2) There are n people who want to get to the top of a building which has only one elevator. You know the weight of each person and the maximum allowed weight in the elevator. What is the minimum number of elevator rides?

Input:

The first input line has two integers n and x: the number of people and the maximum allowed weight in the elevator.

The second line has n integers w₁, w₂, ..., w_n: the weight of each person.

Output:

Print one integer: the minimum number of rides.

Constraints: $1 \leq n \leq 20$ $1 \leq x \leq 10^9$

$$1 \leq w_i \leq x$$

Example Input:

4 10

4 8 6 1

Example Output:

2

Brute Force Approach:

1. Generate all possible subsets of people.
2. For each subset, calculate the total weight.
3. If the total weight is less than or equal to the maximum weight the elevator can carry, count this as a valid ride.
4. Try all combinations of valid rides to find the minimum number of rides needed to transport all people.

Time Complexity: $O(2^n)$, where n is the number of people.

Better Approach (Backtracking):

1. Use backtracking to explore all possible combinations of people that can fit in the elevator.
2. Keep track of the number of rides.
3. If the current group exceeds the weight limit, backtrack and try the next combination.

Time Complexity: Still $O(2^n)$, but typically performs better than brute force due to pruning of impossible combinations.

Optimal Approach (Bitmask Dynamic Programming):

1. Use a bitmask to represent the state of people who have been transported.
2. Iterate over all subsets represented by the bitmask.
3. For each state, calculate the minimum number of rides needed to transport all people represented in the bitmask.
4. Use dynamic programming to save previously computed states to avoid redundant calculations.

Time Complexity: $O(n * 2^n)$, where n is the number of people.

Code:

```
import java.util.*;

public class Q2 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();
        int x = scanner.nextInt();
        int[] weights = new int[n];
        for (int i = 0; i < n; i++) {
            weights[i] = scanner.nextInt();
        }

        int minRides = minimumRides(weights, n, x);
        System.out.println(minRides);
    }
}
```

```
        scanner.close();
    }

    private static int minimumRides(int[] weights, int n, int x) {
        int maxState = 1 << n;
        int[] dp = new int[maxState];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;

        for (int state = 0; state < maxState; state++) {
            for (int i = 0; i < n; i++) {
                if ((state & (1 << i)) == 0) {
                    int nextState = state | (1 << i);
                    int weightSum = 0;
                    for (int j = 0; j < n; j++) {
                        if ((state & (1 << j)) != 0) {
                            weightSum += weights[j];
                        }
                    }
                    weightSum += weights[i];
                    if (weightSum <= x) {
                        dp[nextState] = Math.min(dp[nextState], dp[state]);
                    } else {
                        dp[nextState] = Math.min(dp[nextState], dp[state] + 1);
                    }
                }
            }
        }

        return dp[maxState - 1];
    }
}
```

Test Case 1:

4 10

4 8 6 1

Output:

2

Test Case 2:

5 15

5 7 8 6 4

Output:

3

Test Case 3:

8 25

12 10 5 15 20 5 3 8

Output:

5

Test Case 4:

7 30

10 15 20 5 6 9 8

Output:

4

3) In Domino Solitaire, you have a grid with two rows and many columns. Each square in the grid contains an integer. You are given a supply of rectangular 2×1 tiles, each of which exactly covers two adjacent squares of the grid. You have to place tiles to cover all the squares in the grid such that each tile covers two squares and no pair of tiles overlap.

The score for a tile is the difference between the bigger and the smaller number that are covered by the tile. The aim of the game is to maximize the sum of the scores of all the tiles.

Here is an example of a grid, along with two different tilings and their scores.

<i>Tiling 1</i>				<i>Tiling 2</i>			
8	6	2	3	8	6	2	3
9	7	1	2	9	7	1	2
<i>Score 12</i>				<i>Score 6</i>			

The score for Tiling 1 is $12 = (9-8) + (6-2) + (7-1) + (3-2)$ while the score for Tiling 2 is $6 = (8-6) + (9-7) + (3-2) + (2-1)$. There are other tilings possible for this grid, but you can check that Tiling 1 has the maximum score among all tilings. Your task is to read the grid of numbers and compute the maximum score that can be achieved by any tiling of the grid.

Solution hint:

Recursively find the best tiling, from left to right. You can start the tiling with one vertical tile or two horizontal tiles. Use dynamic programming to evaluate the recursive expression efficiently.

Input format:

The first line contains one integer N , the number of columns in the grid. This is followed by 2 lines describing the grid. Each of these lines consists of N integers, separated by blanks.

Output format:

A single integer indicating the maximum score that can be achieved by any tiling of the given grid.

Algorithm:

1. Read the number of columns N and the values in the top and bottom rows.
2. Initialize a dynamic programming array dp of size N to store the maximum score achievable for each column.
3. Set $dp[0]$ to the absolute difference between the first elements of the top and bottom arrays.
4. For each subsequent column from 1 to $N - 1$:

5. Calculate the score if continuing from the previous column: $dp[i] = dp[i - 1] + \text{abs}(\text{top}[i] - \text{bottom}[i])$.
6. If not the first column, also check the score when skipping the previous column: $dp[i] = \max(dp[i], dp[i - 2] + \text{abs}(\text{top}[i - 1] - \text{top}[i]) + \text{abs}(\text{bottom}[i - 1] - \text{bottom}[i]))$.
7. Output the value in $dp[N - 1]$, which contains the maximum score achievable for the entire grid.

Code:

```
import java.util.Scanner;

public class Q3 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int N = scanner.nextInt();

        int[] top = new int[N];
        int[] bottom = new int[N];

        for (int i = 0; i < N; i++) {
            top[i] = scanner.nextInt();
        }

        for (int i = 0; i < N; i++) {
            bottom[i] = scanner.nextInt();
        }

        int[] dp = new int[N];
        dp[0] = Math.abs(top[0] - bottom[0]);

        for (int i = 1; i < N; i++) {
            dp[i] = dp[i - 1] + Math.abs(top[i] - bottom[i]);
            if (i > 1) {
                dp[i] = Math.max(dp[i], dp[i - 2] + Math.abs(top[i - 1] - top[i]) +
Math.abs(bottom[i - 1] - bottom[i]));
            }
        }

        System.out.println(dp[N - 1]);
        scanner.close();
    }
}
```

Test Case 1:

```
4
8 6 2 3
9 7 1 2
```

Output:

```
12
```

Test Case 2:

12 50

2 4 6 8 10 12 14 16 18 20 22 24

24 22 20 18 16 14 12 10 8 6 4 2

Output:

148

Test Case 3:

20 150

1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39

39 37 35 33 31 29 27 25 23 21 19 17 15 13 11 9 7 5 3 1

Output:

473

Test Case 4:

25 500

100 90 80 70 60 50 40 30 20 10 5 15 25 35 45 55 65 75 85 95 100 105 110 115 120

120 110 100 90 80 70 60 50 40 30 20 10 5 15 25 35 45 55 65 75 85 95 87 98

Output:

825